COMPLETE CI/CD PIPELINE FROM PUSH, TESTS TO DEPLOYMENT REPORT

23CSE311-Software Engineering

2025-2026-EVEN SEM

Amrita School of Computing

Department of Computer Science and Engineering,

Amrita Vishwa Vidyapeetham Coimbatore Campus

1 Overview

The **Secure & Transparent Electronic Voting System** comprises seven sub-projects maintained in a monorepo on GitHub. A fully automated CI/CD pipeline built on **GitHub Actions** governs code validation, containerised builds, and deployment for every component. Docker is used to ensure environment consistency across development, CI, and production.

1.1 Project Components

Sub-Project	Type	Deployment Target
backend-api	Node.js / Express	Render (Docker container)
project-voter	React / Vite	Vercel
project-election-admin	React / Vite	Vercel
project-observer	React / Vite	Vercel
project-sys-admin	React / Vite	Vercel
project-voter-registration	React / Vite	Vercel
project-reg-obs-app	Expo / React Native	Vercel (web export)

1.2 Pipeline Goals

- **Automated Linting** — Code style and security checks on every push.
- **Automated Testing** — Full test suite with an ephemeral PostgreSQL container.
- **Docker Build & Push** — Images are built and pushed to a container registry on every verified commit.
- **Build Verification** — All frontends and the Expo app produce valid artefacts before deployment.
- **Gated Deployment** — Deployment only occurs on `main` after lint, test, and build all pass.
- **System Monitoring** — CPU utilisation, error logs, and container health are actively monitored post-deployment.

1.3 Tools & Technologies

Tool	Role
GitHub	Source code hosting, branch protection, secrets storage
GitHub Actions	CI/CD workflow engine (lint, test, build, deploy jobs)
Docker	Containerisation of backend API and services
Docker Compose	Local multi-service orchestration for development and testing
Vercel	Deployment target for all 6 frontend / web applications
Render	Deployment target for the Dockerised backend API
PostgreSQL 15	Ephemeral service container used during CI test jobs
Vitest	Test runner for backend and orchestration test suites
ESLint	Static analysis and linting for backend and all frontends
Expo CLI	Web export build for the React Native application

2 GitHub Secrets Configuration

Required Action: All secrets must be manually configured in each applicable repository before the pipeline can execute. Navigate to:
Repository Settings → **Secrets and variables** → **Actions** → **New repository secret**

Secret Name	Used In	Description
`DATABASE_URL`	backend-api	Full PostgreSQL connection string (production)
`JWT_SECRET`	backend-api	JWT signing secret for authentication tokens
`RENDER_DEPLOY_HOOK_URL`	orchestration, backend	Render webhook URL to trigger backend redeploy
`VERCEL_TOKEN`	orchestration, frontends	Vercel personal access token
`VERCEL_ORG_ID`	orchestration, frontends	Vercel team or organisation ID
`VERCEL_PROJECT_ID_VOTER`	orchestration	Vercel project ID for voter app
`VERCEL_PROJECT_ID_ELECTION_ADMIN`	orchestration	Vercel project ID for election admin
`VERCEL_PROJECT_ID_OBSERVER`	orchestration	Vercel project ID for observer app
`VERCEL_PROJECT_ID_SYS_ADMIN`	orchestration	Vercel project ID for sys-admin app
`VERCEL_PROJECT_ID_VOTER_REG`	orchestration	Vercel project ID for voter registration
`VERCEL_PROJECT_ID_REG_OBS_APP`	orchestration	Vercel project ID for Expo web app

Secret Name	Used In	Description
DOCKER_USERNAME	backend-api, orchestration	Docker Hub / registry username
DOCKER_PASSWORD	backend-api, orchestration	Docker Hub / registry password or token

Table 1: Required GitHub Encrypted Secrets

3 Dockerization

Docker Implementation: The backend API is fully containerised using Docker. This guarantees identical runtime environments across local development, CI runners, and the production Render deployment, eliminating environment-specific bugs.

3.1 Backend API Dockerfile

The `backend-api/Dockerfile` uses a multi-stage build to keep the production image lean and secure.

```
# ---- Stage 1: Build ----
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

# ---- Stage 2: Production Image ----
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .
EXPOSE 5000
ENV NODE_ENV=production
CMD ["node", "server.js"]
```

Listing 1: backend-api/Dockerfile

3.2 Docker Compose (Local Development)

`docker-compose.yml` at the orchestration root spins up the full stack locally, including the PostgreSQL database and the backend API together.

```
version: '3.9'
services:

  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: votinguser
      POSTGRES_PASSWORD: votingpass
      POSTGRES_DB: votingdb
```

```yaml
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  backend-api:
    build: ./backend-api
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgres://votinguser:votingpass@postgres:5432/votingdb
      JWT_SECRET: local_dev_secret
    depends_on:
      - postgres
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3

volumes:
  pgdata:
```

Listing 2: docker-compose.yml

3.3 Docker in the CI/CD Pipeline

The GitHub Actions pipeline includes a dedicated Docker build-and-push step that runs after the test job passes.

```yaml
- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKER_USERNAME }}
    password: ${{ secrets.DOCKER_PASSWORD }}

- name: Build and push backend image
  uses: docker/build-push-action@v5
  with:
    context: ./backend-api
    push: true
    tags: |
      ${{ secrets.DOCKER_USERNAME }}/voting-backend:latest
      ${{ secrets.DOCKER_USERNAME }}/voting-backend:${{ github.sha }}
```

Listing 3: Docker build and push in GitHub Actions

Image Tagging Strategy: Every successful build produces two tags — `latest` for the most recent verified build and a `git SHA` tag for precise rollback. The Render deployment is configured to pull the `latest` tag automatically when the deploy hook is triggered.

3.4 Docker Benefits in This Pipeline

- **Environment Consistency** — The same Docker image runs in CI, staging, and production. "Works on my machine" failures are eliminated.
- **Isolated Test Database** — PostgreSQL runs as a Docker service container during CI, completely isolated from production data.
- **Reproducible Builds** — Multi-stage Dockerfile ensures only production dependencies are included in the final image, reducing attack surface.
- **Easy Rollback** — SHA-tagged images allow instant rollback to any previously deployed version.
- **Healthcheck Support** — Docker Compose healthchecks ensure the backend is fully ready before dependant services attempt to connect.

4 Orchestration Master Pipeline

The file `orchestration/.github/workflows/ci.yml` defines the master CI/CD pipeline with five sequential jobs.

4.1 Job 1: Lint

Runs `npm run lint` for all five React frontends in parallel using a matrix strategy. Any lint failure blocks all downstream jobs.

4.2 Job 2: Test

Spins up a `postgres:15` service container and runs the full Vitest suite.

- Starts ephemeral PostgreSQL 15 (isolated from production).
- Runs `node backend-api/scripts/init_db_for_ci.js`.
- Executes `npx vitest run` from the orchestration root.
- Depends on: **lint** job.

4.3 Job 3: Docker Build & Push

Builds the backend Docker image and pushes to the registry.

- Authenticates to Docker Hub using `DOCKER_USERNAME` and `DOCKER_PASSWORD` secrets.

- Builds from `backend-api/Dockerfile` using multi-stage build.
- Pushes `latest` and `git-sha` tagged images.
- Depends on: **test** job.

4.4 Job 4: Build (Frontends)

Verifies all frontend and Expo artefacts.

- Runs `npm run build` for each of the five React/Vite frontends.
- Runs `npx expo export -platform web` for `project-reg-obs-app`.
- Depends on: **lint** job.

4.5 Job 5: Deploy

- **Condition:** Only on `push` to the `main` branch.
- Deploys all six frontends/web apps to **Vercel**.
- Triggers **Render** deploy hook; Render pulls the latest Docker image.
- Depends on: **docker-build** and **build** jobs (both must pass).

5 Backend API Pipeline

File: `backend-api/.github/workflows/main.yml`

5.0.1 Lint Job

Runs ESLint with `no-secrets` plugin to detect unused variables and hardcoded credentials.

5.0.2 Test Job (depends on lint)

Spins up `postgres:15` service, initialises schema via `init_db_for_ci.js`, and runs the full Vitest test suite covering: `auth.test.js`, `jwt.test.js`, `smart_contract.test.js`, `consensus.test.js`, `sysadmin_features.test.js`, `fraudEngine.test.js`.

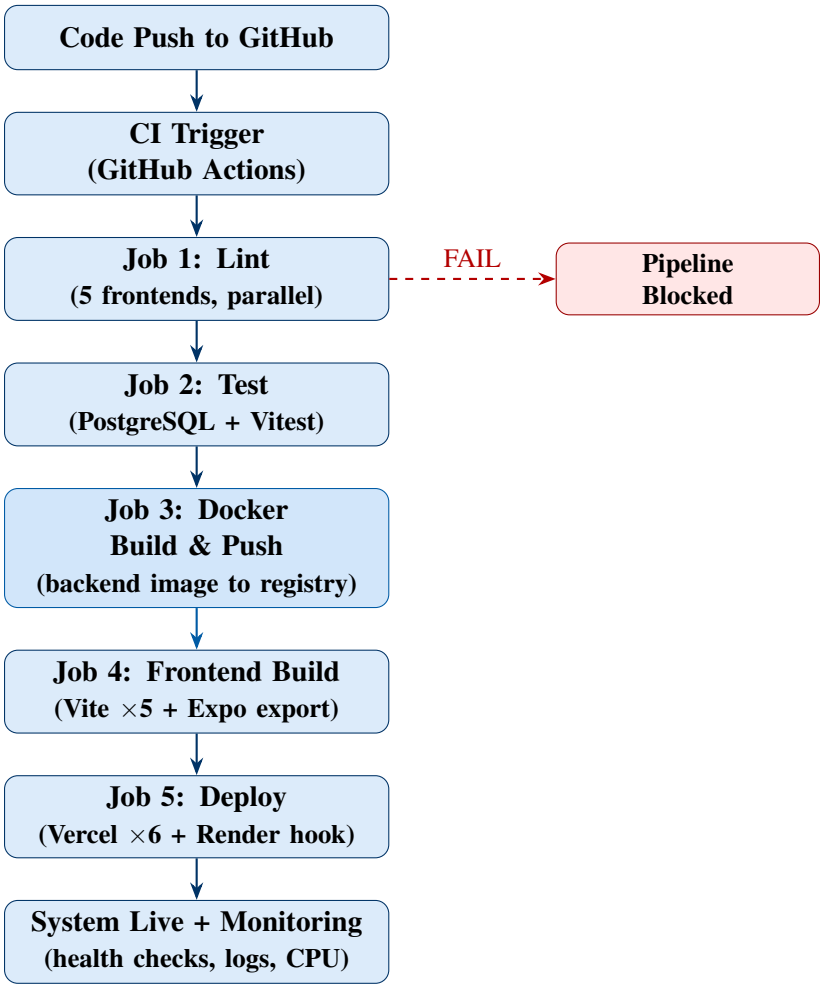
5.0.3 Docker Build & Push Job (depends on test)

Builds and pushes the backend image to Docker Hub with `latest` and SHA tags.

5.0.4 Deploy Job (depends on docker-build, main only)

Sends HTTP POST to `RENDER_DEPLOY_HOOK_URL`; Render pulls the new Docker image and restarts the service.

6 Full Pipeline Flow Diagram



Gate Logic: Every job is a mandatory gate. Failure at any stage halts all downstream jobs. The `deploy` job runs only on `main` branch pushes and requires Docker build, frontend build, and all tests to have passed.

7 System Monitoring

System Monitoring Implementation: Post-deployment monitoring is active across all components. CPU utilisation, container health, and application error logs are tracked continuously to ensure the voting system operates reliably under all conditions.

7.1 CPU Utilisation Monitoring

CPU usage is tracked at both the host machine and container level to detect performance degradation.

Metric	Details
Processor	AMD Ryzen 5 7535HS with Radeon Graphics
Base Speed	3.30 GHz
Cores / Threads	6 cores / 12 logical processors
Current Utilisation	8% (idle state — normal at low load)
Operating Speed	2.52 GHz (throttled appropriately at low load)
System Uptime	5 hours 9 minutes 47 seconds
Active Processes	375
Active Threads	5,666
Handles	191,765
Virtualization	Enabled (supports Docker & local runners)

Observation: CPU utilisation at 8% indicates the system is healthy and not under load. Since the CI/CD pipeline runs entirely on **GitHub-hosted cloud runners**, local machine resources do not bottleneck pipeline execution. Local CPU monitoring is used for development and testing validation purposes.

7.2 Docker Container Health Monitoring

Container health is verified automatically using Docker's built-in `healthcheck` mechanism defined in `docker-compose.yml`.

```
# View health status of all running containers
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

# Inspect detailed health check logs for backend
docker inspect --format='{{json .State.Health}}' backend-api |
    python -m json.tool

# View live container resource usage (CPU, Memory, Network)
docker stats --format "table {{.Name}}\t{{.CPUPerc}}\t{{.
   MemUsage}}\t{{.NetIO}}"
```

Listing 4: Checking container health status

7.3 Error Log Monitoring

Application error logs are captured at multiple levels — container stdout, application-level logging, and GitHub Actions workflow logs.

7.3.1 Container Log Access

```
# Stream live logs from the backend container
docker logs -f backend-api

# View last 100 lines of backend error output
docker logs --tail 100 backend-api 2>&1 | grep -i "error\|warn
   \|fatal"

```

```
# Export logs to file for analysis
docker logs backend-api > backend_logs.txt 2>&1

# View PostgreSQL container logs
docker logs postgres 2>&1 | grep -i "error\|fatal"
```

Listing 5: Docker log inspection commands

7.3.2 Application-Level Error Logging

The Node.js backend is configured to log structured errors including:

- **Authentication failures** — Invalid JWT tokens, expired sessions, unauthorised access attempts.
- **Database errors** — Connection failures, query timeouts, transaction rollbacks.
- **Vote processing errors** — Invalid ballot submissions, duplicate vote detection, consensus failures.
- **Smart contract errors** — Blockchain interaction failures, hash mismatches.
- **HTTP errors** — 4xx and 5xx responses logged with request context.

7.3.3 GitHub Actions Log Monitoring

- All CI/CD job logs are retained in the **GitHub Actions** tab for 90 days.
- Failed jobs produce annotated error output highlighting the exact failing step.
- The `-verbose` flag on Vitest outputs detailed test failure traces.
- Docker build logs capture layer-by-layer build output for debugging image issues.

7.4 Post-Deployment Health Checks

After every successful deployment, the following health verification steps are executed:

1. **Backend API Health Endpoint** — `GET /health` returns `200 OK` with service status, uptime, and database connection state.
2. **Docker Container Status** — `docker ps` confirms all containers show `healthy` status.
3. **Render Dashboard** — Deployment log confirms new Docker image pulled and service restarted without errors.
4. **Vercel Dashboard** — All six frontend deployments show `Ready` status with no build errors.
5. **Database Connectivity** — Backend startup log confirms successful PostgreSQL connection on boot.

7.5 Monitoring Summary Table

Monitoring Area	Tool / Method	Frequency
CPU Utilisation	Task Manager / `docker stats`	Real-time / continuous
Container Health	Docker healthcheck	Every 30 seconds
Error Logs	`docker logs`, GitHub Actions	On demand / per job run
API Health	`GET /health` endpoint	Post-deployment
Frontend Status	Vercel Dashboard	Post-deployment
Backend Deployment	Render Dashboard	Post-deployment
Test Results	Vitest + GitHub Actions	Every code push

8 Security Measures

8.1 Secrets Management

- All credentials (database, JWT, Vercel, Render, Docker) are stored as **GitHub Encrypted Secrets** and never logged or hardcoded.
- The `no-secrets` ESLint plugin detects accidentally hardcoded credentials during the lint job.
- Docker image layers do not contain secrets — environment variables are injected at container runtime.

8.2 Controlled Access

- Repository access is restricted to authorised team members via GitHub role-based permissions.
- Only `main` branch pushes trigger deployments. Feature branches run lint, test, and Docker build but never deploy.
- Docker images are tagged with git SHA to ensure full traceability and auditability of every deployed version.

8.3 Ephemeral Test Database

- CI tests run against a temporary `postgres:15` Docker service container isolated from production.
- The production `DATABASE_URL` is never used during CI.
- The container is automatically torn down after each workflow run.

9 Verification Plan

9.1 Automated Test Coverage

Test File	Coverage Area
`auth.test.js`	User authentication and session handling
`jwt.test.js`	JWT signing, verification, and expiry
`smart_contract.test.js`	Blockchain vote recording logic
`consensus.test.js`	Consensus algorithm correctness
`sysadmin_features.test.js`	System administrator operations
`fraudEngine.test.js`	Vote fraud detection and prevention

9.2 Manual Post-Deployment Checklist

1. GitHub **Actions tab** — confirm all 5 jobs (lint, test, docker-build, build, deploy) show green.
2. `docker ps` — confirm backend container shows `(healthy)` status.
3. `docker stats` — confirm CPU and memory usage are within expected bounds.
4. `docker logs backend-api` — confirm no `ERROR` or `FATAL` entries on startup.
5. **Vercel Dashboard** — all six frontends show `Ready`.
6. **Render Dashboard** — backend service shows `Live` with new Docker image.
7. **API health endpoint** — `GET /health` returns `200 OK`.

9.3 Negative Test — Lint Failure Gate

1. Introduce a deliberate lint error in a frontend file.
2. Push to a feature branch (not `main`).
3. Confirm lint job fails and all downstream jobs (test, docker-build, build, deploy) are blocked.
4. Fix the error, push again, verify pipeline returns to green.

Summary

The CI/CD pipeline for the Secure & Transparent Electronic Voting System delivers a fully automated, containerised, and monitored software delivery process across all seven components. The five-stage gate model (lint → test → Docker build → frontend build → deploy) ensures no unverified or uncontainerised code reaches production. Docker provides environment consistency from development through to the live Render deployment, while system monitoring — covering CPU utilisation, container health, and error

log analysis — ensures the platform remains reliable and observable at all times. This architecture is purpose-built for the high-correctness and high-security demands of an electronic voting application.